

# Acelerador em FPGA para o Supervisor de Segurança

## LiDAR de uma Cadeira de Rodas Semi-Assistida:

Ponto de Controle 1 — Proposta, Especificação e Particionamento

Vítor Gonçalves de Andrade Silva  
Engenharia Eletrônica — FCTE/UnB  
Email: 261025182@aluno.unb.br

Thiago Henrique Oliveira Souza  
Engenharia Eletrônica — FCTE/UnB  
Email: 180131672@aluno.unb.br

**Resumo**—Este primeiro ponto de controle define o escopo de um co-projeto hardware–software sobre Zynq-7000 e justifica, por medição, o bloco a ser acelerado em FPGA. O sistema alvo é uma cadeira de rodas motorizada semi-assistida cujo supervisor de segurança, rodando em Raspberry Pi 5 com ROS 2, decide a cada varredura de LiDAR se o avanço comandado pelo usuário deve ser atenuado. Perfilamos o supervisor e identificamos o teste de corredor — projeção dos 720 feixes e avaliação de 19 direções candidatas, 9918 avaliações por varredura — como o único bloco de custo  $O(N \cdot K)$  e o único cuja latência tem consequência física direta. Propomos um particionamento em que esse custo migra para hardware e a função custo  $O(K)$ , onde residem os ganhos ajustáveis em campo, permanece no ARM. Especificamos a arquitetura — um CORDIC pipelinado compartilhado alimentando 19 pistas paralelas de coeficiente constante —, os formatos de ponto fixo e as interfaces AXI4. Todos os números de custo de hardware apresentados aqui são estimativas analíticas; a implementação RTL, a verificação por equivalência e a síntese são objeto dos próximos pontos de controle. Estabelecemos ainda o protocolo de verificação por modelo de ouro e o cronograma até a entrega final.

### I. INTRODUÇÃO E MOTIVAÇÃO

Cadeiras de rodas motorizadas são inacessíveis a uma parcela expressiva dos usuários que mais precisariam delas: pesquisa clínica clássica aponta que profissionais de reabilitação consideram muitos pacientes inaptos ao uso justamente pela inadequação das interfaces de controle [1]. Espasmos e comandos acidentais no joystick produzem acelerações repentinas e colisões.

O projeto no qual este trabalho se insere converte uma cadeira comercial em um veículo semi-assistido sob um princípio de segurança explícito: o sistema de alto nível pode **frear ou desviar**, mas **nunca acelerar**; a iniciativa de movimento é sempre do usuário. Um supervisor executando em Raspberry Pi 5 com ROS 2 [6] lê um LiDAR RPLIDAR C1 e atenua o comando do joystick antes de repassá-lo ao firmware de tempo real em ESP32-S3. Este co-projeto não reescreve o sistema: escolhe o seu bloco mais crítico e o leva para hardware.

#### A. O bloco a acelerar e por quê

A escolha do bloco não é arbitrária — é resultado de perfilamento do supervisor já existente em software. O teste

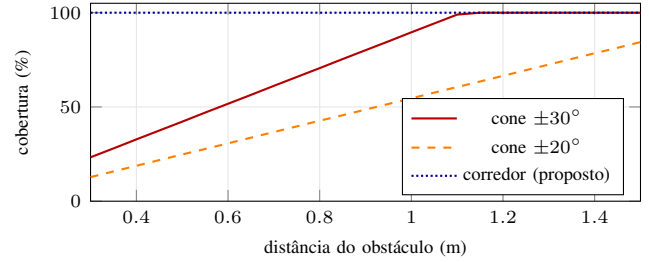


Figura 1. Fração da largura da cadeira efetivamente monitorada. O cone ancorado no sensor *piora* quanto mais perto está o obstáculo, porque estreita enquanto o desvio lateral do vértice permanece constante: a 0,60 m cobre 52 %, e a parcela não coberta é sempre a metade direita. O teste de corredor cobre 100 % por construção. É esta falha que motiva o algoritmo mais caro.

de corredor é, disparadamente, o nó mais pesado: medimos 17,7 % da CPU da Pi 5 na implementação escalar e 8,3 % após vetorização com NumPy. Mais importante que o custo, porém, é a natureza da sua latência: *ela tem consequência física*. O supervisor só corta o comando de avanço; a cadeira ainda percorre uma distância por inércia. Todo atraso entre a chegada da varredura e a decisão se traduz em centímetros adicionais rumo ao obstáculo. Trata-se, portanto, de um caso em que acelerar não é apenas otimização, e sim segurança — o que motiva o co-projeto.

A escolha é ainda mais pertinente porque uma revisão recente do supervisor revelou uma falha geométrica de segurança. A folga frontal era medida por um *cone angular* ancorado no sensor, construção válida apenas se o LiDAR estivesse sobre o eixo longitudinal do veículo. Estando ele 0,336 m fora do eixo, o cone varre uma faixa deslocada e a metade direita da cadeira fica cega justamente nas curtas distâncias (Fig. 1). A correção substitui o cone pelo teste do *corredor físico* que a cadeira ocupará — e é esse algoritmo, correto porém mais caro, que propomos acelerar.

#### B. Trabalhos relacionados

A função custo que pontua direções candidatas pela folga é aparentada ao *Vector Field Histogram* [2], com a distinção essencial de raciocinar sobre a *planta* do robô e não sobre

Tabela I  
PARTICIONAMENTO HW/SW PROPOSTO

| Etapa                        | Custo                 | Onde |
|------------------------------|-----------------------|------|
| Projeção + teste de corredor | $O(N \cdot K) = 9918$ | FPGA |
| Função custo + $\argmin$     | $O(K) = 19$           | ARM  |

um cone do sensor. O CORDIC [3], [4] é a escolha clássica para trigonometria em FPGA sem multiplicadores. O uso de aceleradores acoplados a processadores embarcados via AXI4 sobre Zynq segue a prática consolidada em [5].

## II. ESPECIFICAÇÃO DO ALGORITMO

Cada retorno do LiDAR, um par  $(r_i, a_i)$ , é projetado no referencial `base_link` (origem no centro do eixo traseiro):

$$X_i = x_L + r_i \cos(a_i + \theta_L), \quad Y_i = y_L + r_i \sin(a_i + \theta_L), \quad (1)$$

onde  $(x_L, y_L, \theta_L)$  é a pose calibrada do sensor. Para avaliar um candidato de desvio  $\delta_k$ , giram-se os pontos por  $-\delta_k$  em torno do eixo traseiro — o centro instantâneo de rotação do acionamento diferencial:

$$x_i^{(k)} = X_i \cos \delta_k + Y_i \sin \delta_k, \quad (2)$$

$$y_i^{(k)} = -X_i \sin \delta_k + Y_i \cos \delta_k. \quad (3)$$

O ponto é obstáculo se  $|y_i^{(k)}| \leq W/2$  e  $x_i^{(k)} > X_f$ , com folga  $x_i^{(k)} - X_f$ ; a folga do candidato é o mínimo sobre os pontos. Aqui  $W = 0,61$  m é a largura da cadeira e  $X_f = 0,667$  m a frente do chassi. Note que a estrutura da própria cadeira cai atrás de  $X_f$  e é descartada por construção, sem nenhum limiar arbitrário de distância mínima.

Com  $N = 720$  feixes e  $K = 19$  candidatos ( $\delta_k$  de  $-45^\circ$  a  $+45^\circ$ , passo  $5^\circ$ ), são 9918 avaliações por varredura — o custo  $O(N \cdot K)$  que motiva o acelerador.

## III. PARTICIONAMENTO HARDWARE–SOFTWARE PROPOSTO

O supervisor decompõe-se em duas partes de custo muito distinto (Tab. I). Propomos migrar apenas a primeira para hardware.

A decisão de *não* levar tudo para hardware é deliberada. A função custo são 19 comparações: o ganho seria irrisório. Em compensação, é nela que residem os pesos ajustados em campo ( $w_{\text{obs}}$ ,  $w_{\text{dev}}$ ,  $K_{\text{assist}}$ ), retocados a cada ensaio. Congelá-los em RTL trocaria flexibilidade por nada — exatamente o *trade-off* que o co-projeto existe para arbitrar.

## IV. ARQUITETURA PROPOSTA

A Fig. 2 mostra o *datapath* que pretendemos implementar. O CORDIC é **compartilhado** porque a projeção (1) é comum a todos os candidatos: o custo  $O(N)$  é pago uma única vez. O que se replica são as 19 pistas, que carregam o custo  $O(N \cdot K)$  — e é precisamente aí que esperamos que o paralelismo espacial da FPGA renda: as pistas avaliam *no mesmo ciclo* aquilo que o processador precisa iterar.

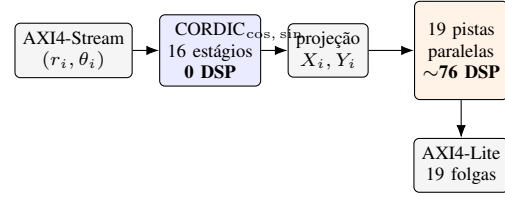


Figura 2. *Datapath* proposto. Vazão alvo de **1 feixe/ciclo**, sem bolhas. O CORDIC é compartilhado (custo  $O(N)$ ); as pistas são replicadas (custo  $O(N \cdot K)$ ). Os números de DSP são *estimativas* a confirmar na síntese (PC3).

Tabela II  
FORMATOS DE PONTO FIXO PROPOSTOS

| Grandeza            | Formato | Faixa         | LSB                       |
|---------------------|---------|---------------|---------------------------|
| distâncias, coords. | Q6.10   | $\pm 32$ m    | 0,98 mm                   |
| cos, sin            | Q1.14   | $\pm 2$       | $6,1 \times 10^{-5}$      |
| ângulos             | Q3.13   | $\pm 4,0$ rad | $1,22 \times 10^{-4}$ rad |

a) **CORDIC**: Optamos por CORDIC em modo rotação [3], pipelinado em 16 estágios, em vez de uma LUT de seno. Uma LUT exigiria uma ROM indexada pelo índice do feixe, o que só funciona se `angle_min`, o incremento e o `yaw` forem fixos em síntese — mas eles são **parâmetros de calibração**, reescritos em tempo de execução por AXI4-Lite. O CORDIC aceita qualquer ângulo e, por usar apenas somadores e deslocamentos cabeados, não deve consumir *nenhum* DSP48. Como o laço só converge para  $|z| \leq 1,7434$  rad, um estágio de redução de quadrante trará  $\theta$  para  $[-\pi/2, \pi/2]$  e inverterá o sinal na saída.

b) **Pistas**: Em cada pista,  $\cos \delta_k$  e  $\sin \delta_k$  serão **constantes de síntese** (*generics*): a rotação (3) vira quatro multiplicações de coeficiente fixo, mapeáveis diretamente em DSP48.

### A. Formato de ponto fixo

Os formatos propostos estão na Tab. II. A escolha da *faixa* — e não apenas da resolução — é um ponto de atenção que pretendemos validar cuidadosamente na verificação de PC2.

### B. Interfaces AXI4

Os feixes chegarão por **AXI4-Stream** (TDATA =  $[\theta(31:16) \mid r(15:0)]$ , TUSER = feixe válido, TLAST = fim da varredura), alimentado por um AXI-DMA que lê a varredura da DDR. O controle e o resultado usarão **AXI4-Lite**: CTRL (*start*), STATUS (*done/busy*) e 19 registradores de folga. Uma interrupção sinalizará a conclusão, dispensando espera ocupada no ARM. A escolha é natural: *stream* para o fluxo volumoso e sequencial (a varredura), mapeado em memória para o punhado de escalares (calibração e resultado).

## V. ESTIMATIVAS ANALÍTICAS DE CUSTO

Como ainda não há RTL nem síntese, os números abaixo são previstos por contagem de operadores e profundidade de *pipeline*; serão confrontados com a medição em PC2 (latência por simulação) e PC3 (recursos e *timing* por síntese).

A latência prevista é da ordem de  $N$  ciclos (um feixe por ciclo) mais a profundidade do *pipeline* do CORDIC e das

Tabela III  
ESTIMATIVAS ANALÍTICAS (A CONFIRMAR EM PC2/PC3)

| Grandeza                  | Estimativa                     | Confirmado em   |
|---------------------------|--------------------------------|-----------------|
| DSP48 (pistas + projeção) | $\sim 78 (19 \times 4 + 2)$    | PC3 (síntese)   |
| DSP48 (CORDIC)            | 0                              | PC3 (síntese)   |
| BRAM                      | 0 ( <i>streaming</i> )         | PC3 (síntese)   |
| Vazão                     | 1 feixe/ciclo                  | PC2 (simulação) |
| Latência                  | $\sim N + \text{profundidade}$ | PC2 (simulação) |
| $F_{\max}$ alvo           | 100 MHz                        | PC3 (síntese)   |

Tabela IV  
CRONOGRAMA ATÉ A ENTREGA FINAL

| Ponto | Entrega                                                | Estado                |
|-------|--------------------------------------------------------|-----------------------|
| PC1   | Proposta, algoritmo, particionamento, arquitetura      | <b>este documento</b> |
| PC2   | RTL, modelo de ouro, verificação, latência             | planejado             |
| PC3   | Síntese, <i>timing</i> , potência, <i>block design</i> | planejado             |

pistas, algo em torno de 750 ciclos por varredura; a 100 MHz, aproximadamente  $7,5 \mu\text{s}$ . Contra o *baseline* de software a medir, isso deve render duas a três ordens de grandeza — hipótese que PC2 verifica.

## VI. PROTOCOLO DE VERIFICAÇÃO E CRONOGRAMA

### A. Verificação por modelo de ouro

Estabelecemos desde já a estratégia de verificação, para que a implementação nasça testável. Não verificaremos por inspeção de formas de onda, e sim por **equivalência bit a bit** contra um modelo de referência:

- 1) um **modelo em ponto fixo** (Python) que emula o RTL exatamente (mesmo CORDIC, mesmos deslocamentos, mesmos truncamentos);
- 2) um **modelo em ponto flutuante** como referência física, para medir o erro de quantização;
- 3) as constantes do RTL (`cordic_pkg.vhd`) serão **geradas** pelo próprio modelo, de modo que RTL e referência casem por construção;
- 4) o estímulo não será sintético: usaremos varreduras **reais** da cadeira, capturadas com um obstáculo no antigo ponto cego.

### B. Cronograma

## VII. CONCLUSÃO PARCIAL

Este ponto de controle fecha a fase de concepção. Justificamos por perfilamento a escolha do teste de corredor como bloco a acelerar, mostramos que sua latência tem consequência física de segurança, especificamos o algoritmo e propusemos um particionamento HW/SW, uma arquitetura CORDIC + pistas e os formatos de ponto fixo e interfaces AXI4. Deliberadamente não apresentamos resultados medidos: as estimativas de recursos, vazão e latência são analíticas e serão confrontadas com implementação e síntese nos próximos pontos de controle. O risco técnico que mais nos preocupa — e que a verificação de PC2 deve expor — é a escolha da faixa dos formatos de ponto fixo, sobretudo a dos ângulos.

## REFERÊNCIAS

- [1] L. Fehr, W. E. Langbein e S. B. Skaar, “Adequacy of power wheelchair control interfaces for persons with severe disabilities: A clinical survey,” *J. Rehabil. Res. Dev.*, vol. 37, no. 3, pp. 353–360, 2000.
- [2] J. Borenstein e Y. Koren, “The Vector Field Histogram — Fast Obstacle Avoidance for Mobile Robots,” *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 278–288, 1991.
- [3] J. E. Volder, “The CORDIC Trigonometric Computing Technique,” *IRE Trans. Electron. Comput.*, vol. EC-8, no. 3, pp. 330–334, 1959.
- [4] R. Andraka, “A survey of CORDIC algorithms for FPGA based computers,” em *Proc. ACM/SIGDA Int. Symp. FPGAs*, 1998, pp. 191–200.
- [5] L. H. Crockett, R. A. Elliot, M. A. Enderwitz e R. W. Stewart, *The Zynq Book: Embedded Processing with the ARM Cortex-A9 on the Xilinx Zynq-7000*. Glasgow: Strathclyde Academic Media, 2014.
- [6] Open Robotics, “ROS 2 Jazzy Jalisco Documentation,” 2024. [Online]. Disponível: <https://docs.ros.org/en/jazzy/>